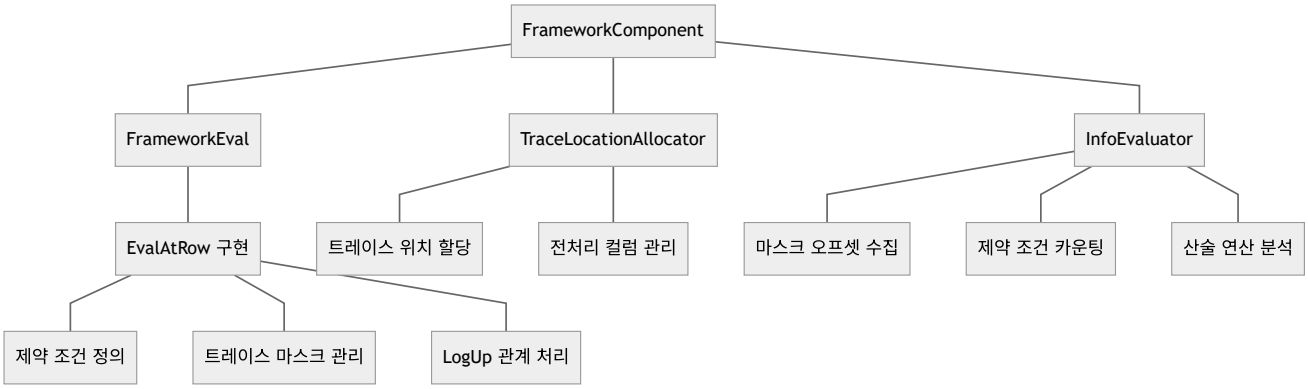
AIR 시스템 - 컴포넌트 기반 증명 시스템 분석

개요

Stwo의 AIR 시스템은 모듈화된 컴포넌트 기반 아키텍처를 통해 복잡한 증명 시스템을 구축합니다. 이 설계는 재사용성, 확장성, 유지보수성을 극대화하여 다양한 계산 패턴을 효율적으로 증명할 수 있게 합니다.

컴포넌트 아키텍처 개요

1. 핵심 컴포넌트 구조



2. 컴포넌트 계층 구조

2.1 추상 컴포넌트 인터페이스

```
pub trait FrameworkEval {
```

```
fn log_size(&self) -> u32;

fn max_constraint_log_degree_bound(&self) -> u32;

fn evaluate<E: EvalAtRow>(&self, eval: E) -> E;

}
```

핵심 특징:

- **크기 정의**: `log_size()` 를 통해 컴포넌트의 트레이스 크기 정의
- **차수 경계**: `max_constraint_log_degree_bound()` 로 제약 조건의 최대 차수 지정
- **평가 로직**: `evaluate()` 에서 실제 제약 조건 정의

2.2 구체적 컴포넌트 구현

```
pub struct FrameworkComponent<C: FrameworkEval> {

eval: C, // 평가 로직

trace_locations: TreeVec<TreeSubspan>, // 트레이스 위치 정보

preprocessed_column_indices: Vec<usize>, // 전처리 컬럼 인덱스

claimed_sum: SecureField, // 주장된 합계

info: InfoEvaluator, // 컴포넌트 정보

}
```

모듈화된 컴포넌트 구조와 재사용성

1. 컴포넌트 템플릿 시스템

1.1 매개변수화된 컴포넌트

```rust
// Fibonacci 컴포넌트 - 길이 N으로 매개변수화
pub struct WideFibonacciEval<const N: usize> {
pub log_n_rows: u32,
}

impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
for _ in 2..N {
let c = eval.next_trace_mask();
// 제약 조건: c = a² + b²
eval.add_constraint(c.clone() - (a.square() + b.square()));
a = b;
b = c;
}
eval
}
}

// 타입 별칭으로 구체적 컴포넌트 정의
```

```
pub type WideFibonacciComponent<const N: usize> = FrameworkComponent<WideFibona
```

1.2 상태 기계 컴포넌트 패밀리

```
// 좌표별로 매개변수화된 상태 전이 컴포넌트
pub struct StateTransitionEval<const COORDINATE: usize> {
pub log_n_rows: u32,
pub lookup_elements: StateMachineElements,
pub claimed_sum: QM31,
}

impl<const COORDINATE: usize> FrameworkEval for StateTransitionEval<COORDINATE>
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let input_state: [_; STATE_SIZE] = std::array::from_fn(|_| eval.next_trace_mask
let mut output_state = input_state.clone();
output_state[COORDINATE] += E::F::one(); // 특정 좌표만 증가
// 입력 상태 룩업
eval.add_to_relation(RelationEntry::new(
&self.lookup_elements,
E::EF::one(),
&input_state,
));
// 출력 상태 룩업 (음의 다중도)
```

```
eval.add_to_relation(RelationEntry::new(
&self.lookup_elements,
-E::EF::one(),
&output_state,
));
eval.finalize_logup_in_pairs();
eval
}
}


// 구체적 컴포넌트 타입들
pub type StateMachineOp0Component = FrameworkComponent<StateTransitionEval<0>>;
pub type StateMachineOp1Component = FrameworkComponent<StateTransitionEval<1>>;
```

2. 컴포넌트 조합 패턴

2.1 컴포넌트 컬렉션

```
pub struct StateMachineComponents {
pub component0: StateMachineOp0Component,
pub component1: StateMachineOp1Component,
}


impl StateMachineComponents {
```

```rust
pub fn components(&self) -> Vec<&dyn Component> {
vec![
&self.component0 as &dyn Component,
&self.component1 as &dyn Component,
]
}
pub fn component_provers(&self) -> Vec<&dyn ComponentProver<SimdBackend>> {
vec![
&self.component0 as &dyn ComponentProver<SimdBackend>,
&self.component1 as &dyn ComponentProver<SimdBackend>,
]
}
}
```

2.2 동적 컴포넌트 관리

```rust
// 증명 생성 시 컴포넌트들을 동적으로 조합
pub fn prove_state_machine(
log_n_rows: u32,
initial_state: State,
config: PcsConfig,
channel: &mut Blake2sChannel,
) -> (StateMachineComponents, StateMachineProof<Blake2sMerkleHasher>) {
// ... 트레이스 생성 ...
```

```
// 컴포넌트 생성
let mut tree_span_provider = &mut TraceLocationAllocator::default();
let component0 = StateMachineOp0Component::new(
tree_span_provider,
StateTransitionEval {
log_n_rows: x_axis_log_rows,
lookup_elements: lookup_elements.clone(),
claimed_sum: claimed_sum_op0,
},
claimed_sum_op0,
);
let component1 = StateMachineOp1Component::new(
tree_span_provider,
StateTransitionEval {
log_n_rows: y_axis_log_rows,
lookup_elements,
claimed_sum: claimed_sum_op1,
},
claimed_sum_op1,
);
let components = StateMachineComponents { component0, component1 };
// 증명 생성
let stark_proof = prove(&components.component_provers(), channel, commitment_sc
```

```
(components, proof)

}
```

컴포넌트 간 상호작용과 데이터 흐름

1. 트레이스 위치 할당 시스템

1.1 TraceLocationAllocator

```
pub struct TraceLocationAllocator {

next_tree_offsets: TreeVec<usize>, // 각 트리의 다음 오프셋

preprocessed_columns: Vec<PreProcessedColumnId>, // 전처리 컬럼들

preprocessed_columns_allocation_mode: PreprocessedColumnsAllocationMode,

}


impl TraceLocationAllocator {

pub fn next_for_structure<T>(

&mut self,

structure: &TreeVec<ColumnVec<T>>,

) -> TreeVec<TreeSubspan> {

TreeVec::new(

zip(&mut *self.next_tree_offsets, &**structure)

.enumerate()
```

```
.map(|(tree_index, (offset, cols))| {
let col_start = *offset;
let col_end = col_start + cols.len();
*offset = col_end;
TreeSubspan { tree_index, col_start, col_end }
})
.collect(),
)
}
}
```

역할:

- **위치 관리**: 각 컴포넌트의 트레이스 컬럼 위치를 자동 할당
- **충돌 방지**: 컴포넌트 간 트레이스 위치 충돌 방지
- **전처리 컬럼**: 공유되는 전처리 컬럼의 일관된 관리

1.2 컴포넌트 생성 과정

```
impl<E: FrameworkEval> FrameworkComponent<E> {
pub fn new(
location_allocator: &mut TraceLocationAllocator,
eval: E,
claimed_sum: SecureField,
) -> Self {
// 1. 정보 수집
```

```
let info = eval.evaluate(InfoEvaluator::new(eval.log_size(), vec![], claimed_su

// 2. 트레이스 위치 할당

let trace_locations = location_allocator.next_for_structure(&info.mask_offsets)

// 3. 전처리 컬럼 인덱스 매핑

let preprocessed_column_indices = info

.preprocessed_columns

.iter()

.map(|col| {

// 기존 컬럼 찾기 또는 새로 추가

if let Some(pos) = location_allocator

.preprocessed_columns

.iter()

.position(|x| x.id == col.id)

{

pos

} else {

location_allocator.preprocessed_columns.push(col.clone());

location_allocator.preprocessed_columns.len() - 1

}

})

.collect();

Self {

eval,
```

```
trace_locations,
info,
preprocessed_column_indices,
claimed_sum,
}
}
}
```

2. 정보 수집 및 분석 시스템

2.1 InfoEvaluator

```
pub struct InfoEvaluator {
pub mask_offsets: TreeVec<Vec<Vec<isize>>>, // 마스크 오프셋 정보
pub n_constraints: usize, // 제약 조건 수
pub preprocessed_columns: Vec<PreProcessedColumnId>, // 전처리 컬럼들
pub logup: LogupAtRow<Self>, // LogUp 정보
pub arithmetic_counts: ArithmeticCounts, // 산술 연산 카운트
pub logup_counts: LogupCountPerRow, // LogUp 카운트
}

impl EvalAtRow for InfoEvaluator {
type F = FieldCounter;
type EF = ExtensionFieldCounter;
```

```
fn next_interaction_mask<const N: usize>(
&mut self,
interaction: usize,
offsets: [isize; N],
) -> [Self::F; N] {
// 마스크 오프셋 정보 수집
if self.mask_offsets.len() <= interaction {
self.mask_offsets.resize(interaction + 1, vec![]);
}
self.mask_offsets[interaction].push(offsets.into_iter().collect());
array::from_fn(|_| FieldCounter::one())
}
fn add_constraint<G>(&mut self, constraint: G) {
// 제약 조건 수 카운팅 및 산술 연산 분석
let lin_combination = ExtensionFieldCounter::one() + ExtensionFieldCounter::one
self.arithmetic_counts.merge(lin_combination.drain());
self.n_constraints += 1;
}
}
```

2.2 산술 연산 분석

```
#[derive(Debug, Default, Clone, Copy, PartialEq, Eq)]
```

```rust
pub struct ArithmeticCounts {
pub n_ef_mul_ef: usize, // EF * EF 연산 수
pub n_ef_mul_f: usize, // EF * F 연산 수
pub n_ef_mul_base_field: usize, // EF * BaseField 연산 수
pub n_ef_add_ef: usize, // EF + EF 연산 수
pub n_ef_add_f: usize, // EF + F 연산 수
pub n_ef_add_base_field: usize, // EF + BaseField 연산 수
pub n_f_mul_f: usize, // F * F 연산 수
pub n_f_mul_base_field: usize, // F * BaseField 연산 수
pub n_f_add_f: usize, // F + F 연산 수
pub n_f_add_base_field: usize, // F + BaseField 연산 수
}
```

3. 관계 추적 시스템 (Relation Tracking)

3.1 관계 추적 메커니즘

```rust
pub fn track_state_machine_relations(
trace: &TreeVec<Vec<&CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>>
StateMachineComponents { component0, component1 }: &StateMachineComponents,
) -> Vec<RelationTrackerEntry> {
let trace = trace.as_ref().map_cols(|col| col.to_cpu().values);
let trace = &trace.as_cols_ref();
```

```
chain!(
add_to_relation_entries(component0, trace),
add_to_relation_entries(component1, trace)
)
.collect()
}


pub struct RelationTrackerEntry {
pub relation_name: String,
pub values: Vec<BaseField>,
pub multiplicity: i32,
}
```

3.2 관계 요약 시스템

```
impl RelationSummary {
pub fn summarize_relations(entries: &[RelationTrackerEntry]) -> Self {
let mut summary = HashMap::new();
for entry in entries {
let relation_entries = summary
.entry(entry.relation_name.clone())
.or_insert_with(Vec::new);
// 동일한 값들을 가진 항목들을 병합
```

```
if let Some(existing) = relation_entries
.iter_mut()
.find(|(values, _)| *values == entry.values)
{
existing.1 += entry.multiplicity;
} else {
relation_entries.push((entry.values.clone(), entry.multiplicity));
}
}
Self { summary }
}
}
```

확장성과 유지보수성을 위한 설계 원칙

1. 인터페이스 분리 원칙

1.1 역할별 트레이트 분리

```
// 컴포넌트 정의를 위한 트레이트
pub trait FrameworkEval {
fn log_size(&self) -> u32;
fn max_constraint_log_degree_bound(&self) -> u32;
fn evaluate<E: EvalAtRow>(&self, eval: E) -> E;
```

```
}

// 제약 조건 평가를 위한 트레이트
pub trait EvalAtRow {
type F: FieldExpOps + /* ... */;
type EF: One + Clone + /* ... */;
fn next_trace_mask(&mut self) -> Self::F;
fn add_constraint<G>(&mut self, constraint: G);
fn add_to_relation<R: Relation<Self::F, Self::EF>>(&mut self, entry: RelationE
}

// 관계 정의를 위한 트레이트
pub trait Relation<F: Clone, EF: RelationEFTraitBound<F>>: Sized {
fn combine(&self, values: &[F]) -> EF;
fn get_name(&self) -> &str;
fn get_size(&self) -> usize;
}
```

1.2 백엔드 추상화

```
// SIMD 백엔드용 컴포넌트 증명자
impl<E: FrameworkEval + Sync> ComponentProver<SimdBackend> for FrameworkComponen
fn evaluate_constraint_quotients_on_domain(
```

```
&self,
trace: &Trace<'_, SimdBackend>,
evaluation_accumulator: &mut DomainEvaluationAccumulator<SimdBackend>,
) {
// SIMD 최적화된 제약 조건 평가
if trace_domain.log_size() < LOG_N_LANES + LOG_N_VERY_PACKED_ELEMS {
// CPU 폴백
let mut col = accum.col.to_cpu();
for row in 0..(1 << eval_domain.log_size()) {
let eval = CpuDomainEvaluator::new(/* ... */);
let row_res = self.eval.evaluate(eval).row_res;
col.set(row, col.at(row) + row_res * denom_inv);
}
} else {
// SIMD 처리
let col = unsafe { VeryPackedSecureColumnByCoords::transform_under_mut(accum.co
col.chunks_mut(CHUNK_SIZE).for_each(|mut chunk| {
let eval = SimdDomainEvaluator::new(/* ... */);
let row_res = self.eval.evaluate(eval).row_res;
// SIMD 연산으로 결과 업데이트
});
}
}
```

```
}
```

2. 의존성 주입 패턴

2.1 평가자 주입

```
impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
// 평가자 E는 외부에서 주입됨
// 이를 통해 다양한 평가 전략 사용 가능:
// - InfoEvaluator: 정보 수집
// - ExprEvaluator: 표현식 생성
// - CpuDomainEvaluator: CPU 도메인 평가
// - SimdDomainEvaluator: SIMD 도메인 평가
// - AssertEvaluator: 제약 조건 검증
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
for _ in 2..N {
let c = eval.next_trace_mask();
eval.add_constraint(c.clone() - (a.square() + b.square()));
a = b;
b = c;
}
eval
```

```
}

}
```

2.2 백엔드 주입

```
// 다양한 백엔드에서 동일한 컴포넌트 사용 가능

pub fn generate_trace<const N: usize>(

log_size: u32,

inputs: &[FibInput],

) -> ColumnVec<CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>> {

// SIMD 백엔드 사용

let mut trace = (0..N)

.map(|_| Col::<SimdBackend, BaseField>::zeros(1 << log_size))

.collect_vec();

// ... 트레이스 생성 로직 ...

}


// CPU 백엔드로도 동일한 로직 사용 가능

// pub fn generate_trace_cpu<const N: usize>(...) -> ColumnVec<CircleEvaluation<
```

3. 컴포지션 패턴

3.1 컴포넌트 합성

```rust
// 여러 컴포넌트를 조합하여 복잡한 시스템 구성
pub struct ComplexSystem {
fibonacci_component: WideFibonacciComponent<100>,
poseidon_component: PoseidonComponent,
state_machine_components: StateMachineComponents,
}

impl ComplexSystem {
pub fn prove(&self, channel: &mut impl Channel, commitment_scheme: Commitment
let all_components = vec![
&self.fibonacci_component as &dyn ComponentProver<SimdBackend>,
&self.poseidon_component as &dyn ComponentProver<SimdBackend>,
];
all_components.extend(self.state_machine_components.component_provers());
prove(&all_components, channel, commitment_scheme).unwrap()
}
}
```

3.2 관계 공유

```rust
// 여러 컴포넌트가 동일한 관계를 공유할 수 있음
relation!(SharedLookupTable, 4);
```

```rust
pub struct ComponentA {
shared_relation: SharedLookupTable,
}

pub struct ComponentB {
shared_relation: SharedLookupTable, // 동일한 관계 공유
}

impl FrameworkEval for ComponentA {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let values = [/* ... */];
eval.add_to_relation(RelationEntry::new(
&self.shared_relation,
E::EF::one(),
&values,
));
eval
}
}
```

4. 테스트 가능성

4.1 단위 테스트 지원

```rust
#[cfg(test)]
mod tests {
use stwo_constraint_framework::{assert_constraints_on_polys, ExprEvaluator};
#[test]
fn test_fibonacci_constraints() {
const LOG_N_INSTANCES: u32 = 6;
let traces = TreeVec::new(vec![vec![], generate_test_trace(LOG_N_INSTANCES)]);
let trace_polys = traces.map(|trace| trace.into_iter().map(|c| c.interpolate())


assert_constraints_on_polys(
&trace_polys,
CanonicCoset::new(LOG_N_INSTANCES),
|assert_eval| {
WideFibonacciEval::<100> { log_n_rows: LOG_N_INSTANCES }.evaluate(assert_eval);
},
SecureField::zero(),
);
}
#[test]
fn test_constraint_expression_generation() {
let component = WideFibonacciEval::<3> { log_n_rows: 4 };
let eval = component.evaluate(ExprEvaluator::new());
```

```
// 생성된 제약 조건 표현식 검증
assert_eq!(eval.constraints.len(), 1);
println!("Generated constraint: {}", eval.format_constraints());
}
}
```

4.2 성능 분석 지원

```
#[test]
fn test_arithmetic_analysis() {
let component = PoseidonEval { /* ... */ };
let info = component.evaluate(InfoEvaluator::empty());
println!("Arithmetic counts: {:?}", info.arithmetic_counts);
println!("LogUp counts: {:?}", info.logup_counts);
println!("Number of constraints: {}", info.n_constraints);
}
```

성능 최적화 전략

1. 컴파일 타임 최적화

1.1 제네릭 특화

```
// 컴파일 타임에 특화된 코드 생성
```

```rust
impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
// N이 컴파일 타임 상수이므로 루프가 언롤됨
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
for _ in 2..N { // 컴파일 타임에 언롤됨
let c = eval.next_trace_mask();
eval.add_constraint(c.clone() - (a.square() + b.square()));
a = b;
b = c;
}
eval
}
}
```

1.2 타입 레벨 최적화

```rust
// 타입 시스템을 통한 최적화
pub struct StateTransitionEval<const COORDINATE: usize> {
// COORDINATE가 컴파일 타임 상수이므로 분기 제거
}

impl<const COORDINATE: usize> FrameworkEval for StateTransitionEval<COORDINATE>
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
```

```
let mut output_state = input_state.clone();
output_state[COORDINATE] += E::F::one(); // 컴파일 타임에 인덱스 결정
// ...
}
}
```

2. 런타임 최적화

2.1 SIMD 활용

```
impl<E: FrameworkEval + Sync> ComponentProver<SimdBackend> for FrameworkComponen
fn evaluate_constraint_quotients_on_domain(&self, /* ... */) {
// SIMD 처리를 위한 청크 단위 처리
let col = unsafe { VeryPackedSecureColumnByCoords::transform_under_mut(accum.co
#[cfg(feature = "parallel")]
let iter = range.into_par_iter().step_by(CHUNK_SIZE).zip(col.chunks_mut(CHUNK_S
iter.for_each(|(chunk_idx, mut chunk)| {
// SIMD 최적화된 평가
let eval = SimdDomainEvaluator::new(/* ... */);
let row_res = self.eval.evaluate(eval).row_res;
unsafe {
let denom_inv = VeryPackedBaseField::broadcast(denom_inv[vec_row >> offset]);
chunk.set_packed(idx_in_chunk, chunk.packed_at(idx_in_chunk) + row_res * denom_i
}
```

```
});
}
}
```

2.2 병렬 처리

```
// Rayon을 활용한 병렬 처리
#[cfg(feature = "parallel")]
let iter = range.into_par_iter().step_by(CHUNK_SIZE).zip(col.chunks_mut(CHUNK_S

#[cfg(not(feature = "parallel"))]
let iter = range.step_by(CHUNK_SIZE).zip(col.chunks_mut(CHUNK_SIZE));
```

결론

Stwo의 컴포넌트 기반 AIR 시스템은 다음과 같은 핵심 장점을 제공합니다:

1. 모듈성과 재사용성

- **템플릿 기반 설계**: 매개변수화된 컴포넌트로 다양한 변형 지원
- **인터페이스 분리**: 역할별로 분리된 트레이트로 유연한 조합 가능
- **컴포지션 패턴**: 복잡한 시스템을 단순한 컴포넌트들의 조합으로 구성

2. 확장성

- **동적 컴포넌트 관리**: 런타임에 컴포넌트 조합 가능

- **백엔드 추상화**: 다양한 하드웨어 백엔드 지원
- **관계 공유**: 컴포넌트 간 효율적인 데이터 공유

3. 유지보수성

- **의존성 주입**: 테스트와 디버깅이 용이한 구조
- **정보 수집 시스템**: 자동화된 성능 분석과 최적화
- **표현식 생성**: 제약 조건의 시각적 검증 가능

4. 성능

- **컴파일 타임 최적화**: 제네릭 특화와 루프 언롤링
- **SIMD 활용**: 벡터화된 연산으로 고성능 달성
- **병렬 처리**: 멀티코어 활용으로 확장성 확보

이러한 설계를 통해 Stwo는 복잡한 Circle STARK 증명 시스템을 효율적이고 유지보수 가능한 방식으로 구현할 수 있습니다.